

Homework 5

Gabriela Rubio

March 31, 2026

1. (a) **Algorithm Description:** We can build this algorithm by defining an array of booleans $f[0...M]$, $f[j]$ being true if there exists a combination of items with repetition allowed that the sizes of such items sum to be exactly j . The base case is $f[0] = \text{true}$, and we start with $f[j] = \text{false}$ for all j that is bigger than 0. For each j from 1 to M , we can check every item a_i such that if $j \geq a_i$ and $f[j - a_i]$ is true, then $f[j] = \text{true}$. The final answer is $f[M]$.

(b) **Pseudocode:**

```
Algorithm: UnlimitedKnapsack
Input: n item sizes a[1..n] and a knapsack size M
Output: true if the items sum to exactly M, false otherwise

f[0] = true
for j = 1 to M:
    f[j] = false

for j = 1 to M:
    for i = 1 to n:
        if j >= a[i] AND f[j - a[i]] == true:
            f[j] = true
            break

return f[M]
```

- (c) **Correctness:** The algorithm is correct because it follows the steps of designing a dynamic programming algorithm. The subproblem is finding $f[j]$, and it represents whether it is possible to fill a knapsack of capacity j using any combination of the given items. The base case sets $f[0] = \text{true}$ since an empty subset will sum to 0. The for loop iterates from $j = 1$ to M for each j , and we check every item a_1 to see if it can be the last item added to reach j . In our dependency relation, $f[j] = \text{true}$ if there exists an item a_i such that $j \geq a_i$ and $f[j - a_i] = \text{true}$. Each $f[j]$ depends on the earlier entries since we do $f[j - a_i]$, where $j - a_i < j$. The goal is to find $f[M]$, which is the answer to whether it is possible to fill a knapsack of size M entirely.

- (d) **Time Analysis:** The outer loop in the algorithm runs until M , and the inner loop runs until n , so the time complexity is $O(nM)$. Also, the array uses $O(M)$ space.

2. (a) **Algorithm Description:** We can solve this problem by making a 2D array $f[0..n][0..M]$, and $f[i][j]$ being the maximum value possible using only the first i items for a knapsack of capacity j . For each i and capacity j , we use i if it fits or skip it if it doesn't. Then, the final answer is $f[n][M]$.

(b) **Pseudocode:**

```
Algorithm: KnapsackWithWeights
Input: n items with sizes a[1..n] and values v[1..n],
and a knapsack of capacity M
Output: maximum total weight achievable with the given knapsack

for j = 0 to M:
    f[0][j] = 0
```

```

for i = 1 to n:
    for j = 0 to M:
        f[i][j] = f[i-1][j]
        if j >= a[i]:
            f[i][j] = max(f[i][j],
                          f[i-1][j - a[i]] + v[i])

return f[n][M]

```

- (c) **Correctness:** The algorithm is correct because it follows the steps of designing a dynamic programming algorithm. The subproblem is finding $f[i][j]$, which is the maximum total weight achievable using only the items $[1..i]$ with a capacity of at most j . The base case is $f[0][j] = 0$ for all of j since the value will always be 0 if no items are being used. The for loop iterates i to 1 to n , and j from 0 to M , so it fills the "table" row by row. IN our dependency relation, $f[i][j]$ depends only on the row $i-1$ which is the previous item already computed. We can either skip i and use $f[i-1][j]$ or take the item i and add $v[i]$ to $f[i-1][j - a[i]]$. The goal is to find $f[n][M]$, which is the max weight using all n items for the knapsack M .
- (d) **Time Analysis:** We use, again, two nested loops. The outer one runs n times, and the inner one runs M times, so the time complexity is $O(nM)$. Since we use a 2D array, the space is also $O(nM)$.
3. (a) **Algorithm Description:** To solve this we use a 2D array $f[0..n][0..m]$, such that $f[i][j]$ is the maximum sum from a common subsequence of $A[1..i]$ and $B[1..j]$. If $A[i] == B[j]$, we can include the element and add the value or skip it. If they're not equal, we skip one of them. The final answer will be $f[n][m]$.
- (b) **Pseudocode:**

```

Algorithm: MaxSumCommonSubsequence
Input: Arrays A[1..n] and B[1..m]
Output: The maximum sum of any common subsequence of A and B

for i = 0 to n:
    f[i][0] = 0

for j = 0 to m:
    f[0][j] = 0

for i = 1 to n:
    for j = 1 to m:
        if A[i] == B[j]:
            f[i][j] = max(f[i - 1][j - 1] + A[i],
                          f[i - 1][j],
                          f[i][j - 1])
        else:
            f[i][j] = max(f[i - 1][j], f[i][j - 1])

return f[n][m]

```

- (c) **Correctness:** The algorithm is correct because it follows the steps of designing a dynamic programming algorithm. The subproblem is finding $f[i][j]$, which is the maximum sum of any common subsequence of $A[1..i]$ and $B[1..j]$. The base case is $f[0] = f[0][j] = 0$ for all i and j , because an empty array has no common subsequence and the maximum sum is 0. The for loop runs i from 1 to n , and j from 1 to m , and it fills the "table" row by row. For the dependency relation, $f[i][j]$ depends on $f[i-1][j-1]$, $f[i-1][j]$ and $f[i][j-1]$, which are solved before $f[i][j]$. If $A[i] == B[j]$ we take the max of either including or skipping the element, and if they are not equal, we skip either $A[i]$ or $B[j]$. The goal is finding $f[n][m]$ which is the maximum sum commom subsequence of A and B .

- (d) **Time Analysis:** We use two nested loops for the algorithm. The outer one runs n times, and the inner one runs m times, so the time complexity is $O(nm)$ and the space as well.
4. (a) **Algorithm Description:** We use an array $f[1..n]$, where $f[i]$ is the length of the longest increasing subsequence that ends at $A[i]$. For each i , we look back at all $j < i$, and if $A[j] < A[i]$ we extend the subsequence. We use another array $p[1..n]$ to reconstruct the actual subsequence, and the answer is the index i with the maximum $f[i]$.

(b) **Pseudocode:**

```

Algorithm: LongestIncreasingSubsequence
Input: Array A[1..n] of distinct numbers
Output: the longest increasing subsequence of A, with no repeating numbers

for i = 1 to n:
    f[i] = 1
    p[i] = -1

for i = 1 to n:
    for j = 1 to i - 1:
        if A[j] < A[i] and f[j] + 1 > f[i]:
            f[i] = f[j] + 1
            p[i] = j

best = 1

for i = 2 to n:
    if f[best] < f[i]:
        best = i

result = {}
x = best
while x != -1:
    push A[x] to result
    x = p[x]

return result

```

- (c) **Correctness:** The algorithm is correct because it follows the steps of designing a dynamic programming algorithm. The subproblem is finding $f[i]$, which is the length of the longest increasing subsequence that ends at index i . The base case is $f[i] = 1$ for all i , because a single element is a subsequence of length 1. The for loop iterates i from 1 to n , and for each i we look back at all $j < i$ to then find the best previous element to extend the subsequence. For the dependency relation, $f[i]$ depends on $f[j]$ for each $j < i$, which are computed beforehand. We also set $f[i] = f[j] + 1$ whenever $A[j] < A[i]$ to make sure we get the correct best index. The goal is finding the longest increasing subsequence, which, after finding the best, is reconstructed by finding the $f[\text{best}]$, and then following the pointers back to the start.
- (d) **Time Analysis:** The algorithm uses two nested loops. The outer one runs n time and the inner one runs n time, so we get $O(n^2)$ time, and the space used is $O(n)$.